DEREXI / CYBER THREATS & VULNERABILITIES II

Enterprise Vulnerability Management

ASSIGNMENT 02 | TECHNICAL DELIVERY & EVIDENCE REPORT

A documented security program, reconciled asset map, automated control testing, custom vulnerability inspector, and evidence-led remediation process.

33	13	5
Security checks	Local assets	Findings open

DeRexi Policy Pilot • Fictional case study: Aurum Capital Bank

Evidence captured November 9, 2026 | Report compiled November 10, 2026

Local-lab assessment; production authorization, enterprise deployment and remediation are not established.

01 / ORIENTATION

Executive overview

What was accomplished, what the numbers mean, and where the evidence stops.

This assignment develops a **repeatable vulnerability management program** around the DeRexi application rather than presenting an isolated scan. The team model, asset relationships, signed access-control tests, Python dependency review, custom scoring tool and evidence register form one local discovery-to-retest cycle.

27/33	46.15%	9/9
Controls passing	Direct asset coverage	Inspector regression

The assessment in one minute

• **13 of 13 in-scope assets reconciled:** each has an evidence source, proposed owner, dependencies, business impact and documented limitation. This is not 100% enterprise discovery.

• **33 checks executed:** 27 passed, six failed, with zero harness errors in the final rerun. A failed check is a valid security finding, not an invalid assessment.

• **35 Python package versions checked:** all registry queries succeeded; four advisory records consolidate to two distinct affected packages.

• **Five findings remain OPEN:** one Critical, two High, two Medium on the documented local risk scale. None is closed or excepted.

> **Overall assessment:** the documented local workflow and testing deliverables are demonstrated. The rubric's unrestricted **enterprise** requirement remains **PARTIAL** until deployment scope, accountable ownership, recurring operation and post-fix testing are established.

How to read this report

Sections 02–07 explain each deliverable in ordinary language, link it to the actual evidence, and state what the observed test means. Sections 08–10 present the open findings, remediation playbook, rubric matrix and artifact index.

02 / GOVERNANCE

The vulnerability management program

A repeatable operational process, with proposed owners and explicit acceptance criteria.

Vulnerability management is the process of **finding, validating, prioritizing, correcting and re-testing** weaknesses. DeRexi's **program.md** formalizes those stages, evidence requirements, roles, escalation timing and review cadence. Only one local cycle was executed; recurring bank operations are proposed, not evidenced.

Program flow — from signal to closure

01 DISCOVER	02 VALIDATE	03 PRIORITIZE	04 TRACK	05 RETEST
Asset/source change, advisory or test result	Prove behavior and capture observed writes / error codes	Calculate likelihood × impact (1–5 each)	Open owner queue; evidence, target and treatment	Repeat the failed negative test plus valid positive test

Functional accountability — no roles or accounts were provisioned

Proposed function	Responsibility
Security assessment	Independent validation, reproducible evidence and retest
Platform engineering	Application API, libraries, Docker and remediation coordination
Identity engineering	Signed JWT validation, privileged roles and user boundaries
Data platform	PostgreSQL data integrity, RLS and recovery assurance
Feed owners	Regulatory publisher attribution and source resilience
Business / compliance owners	Impact decisions, treatment approval and exception acceptance

Prioritization and escalation targets

Priority	Local score	Program target
Critical	16–25	72 hours from validated intake
High	10–15	14 days
Medium	5–9	30 days
Low	1–4	90 days

Active exploitation triggers immediate containment and incident escalation. Exceptions require documented justification, compensating controls, a real approver and expiry; the record shows **zero approved exceptions**. The team structure and service levels are documented proposals, not signed organizational commitments.

03 / ASSET INTELLIGENCE

Advanced asset discovery & relationship mapping

Inventory is useful only when it connects components to the business risks they carry.

The original register lists **14 assets**: 13 local DeRexi ecosystem components plus one historical classroom SSH host. The SSH host is excluded from this application denominator. Reconciliation verified 13/13 evidence records; signed or technical control assertions directly assessed only **6/13**.

DeRexi dependency pathways

Employee interface → Policy API → Identity / Database	A policy-write flaw may change guidance employees rely on.
Signed JWT / roles → Database role resolution	Privilege errors can expose or alter restricted information.
Government feeds → Pulse normalization → Docker dashboard	False publisher attribution can mislead policy owners.
Python packages → API / identity issuer	Library drift and vulnerable releases can undermine trust.
Optional AI provider ← Application context	Potential disclosure or inaccurate guidance; provider not contacted.

Critical application assets and evidence limits

Asset	Criticality	Assessment basis / limit
APP — Policy & incident API	5 / 5	Signed endpoint fixtures; fake writes, not live database
AUTH — JWT and roles	5 / 5	Local signed identities; production JWKS not verified
DB — Hosted PostgreSQL design	5 / 5	Schema/source reviewed; live RLS, restore, row counts not tested
PULSE — Feed normalization	4 / 5	Source inspection and hostile-host negative assertion
PKGS — Local packages	4 / 5	35-distribution Windows environment; not deployed image

Two different coverage measures: 100% of the approved local register was reconciled; **46.15%** had direct current control tests. Neither figure proves the bank's full asset population was discovered.

Prior classroom Nmap evidence (10 active hosts and one detailed host in the submitted closure document) remains a separate historical population; raw host XML and bank-wide ownership are not available.

04 / AUTOMATION

Automated security assessment framework

What the runner actually checks — and why the results are reproducible.

The **run_assessment.py** orchestration executes a local control suite, captures installed Python dependency consistency, and generates timestamped machine evidence. **control_suite.py** includes signed requests, negative authorization cases and static evidence checks. It uses an isolated fake session rather than writing to live Supabase.

33	27	6
Executed checks	Pass	Fail

Six reproduced failing assertions

Check	Expected	Observed	Security interpretation
EMP-APPROVE	403	201	Employee could create an approved policy
OWN-APPROVE	403	201	Owner could approve against admin-only test rule
ANON-APPROVE	401	201	Unsigned request could create an approved policy
EMP-VERSION	403	201	Employee could publish a policy version
FEED-HOST	Reject URL	Accepted	Publisher label not bound to actual FDIC host
DEPENDENCY-PINS	Reproducible lock	Lower bounds	Version drift is possible without a reviewed lock

What also passed

Other signed-identity cases included legitimate permitted operations, cross-user access denial and altered-signature rejection. Input rejection, API headers and container configuration checks also contributed to the passing set. The closure validation confirmed **the same 33 check identities and outcomes** on the independent rerun.

The final run was **20261109T125144008620Z**. Earlier incomplete or sandbox-limited attempts were retained as failures/errors, not silently counted as successful coverage. The final six failures are security-control outcomes, not test-tool crashes.

05 / CUSTOM TOOLING

DeRexi Vulnerability Inspector

A purpose-built defensive CLI that converts raw evidence into prioritized action.

The custom **derexi_vulnerability_inspector.py** is an offline evidence-to-findings tool. It reads the approved asset register, control results, dependency snapshot and optional advisory triage; groups related failures; calculates local risk scores; and exports machine-readable JSON and human-readable Markdown.

Input → analysis → output

INPUTS	ANALYSIS	DELIVERABLES
assets.json + results.json + dependencies.json + optional advisories-triaged.json	Coverage denominator; failure grouping; impact × likelihood; severity; owner and SLA	findings.json / findings.md, metrics and OPEN remediation queue

Command-line interface (documented contract)

```
python derexi_vulnerability_inspector.py
--assets assets.json --results results.json --dependencies dependencies.json
--advisories advisories-triaged.json --output <output-directory>
```

Regression evidence

Scenario family	Outcome
Repeated failures are grouped without inflating findings	PASS
Historical assets excluded from current coverage denominator	PASS
Harness errors retained; no fabricated closure	PASS
Risk boundary values and malformed records rejected	PASS
Advisory aliases deduplicated; snapshots matched	PASS
All nine recorded tool regression scenarios	9 / 9 PASS

The first regression attempt failed due to a Windows Codex temporary-directory permission error. Running the same assertions with a workspace-local temporary directory yielded **nine passes**. Both logs are preserved; the permissions failure was not a product-control remediation.

06 / DEPENDENCIES

Software supply-chain & advisory review

A point-in-time survey of the exact Windows Python environment—not a deployed vulnerability scan.

The source package scan queried the public PyPI release-specific JSON vulnerability records for **35/35 installed distributions** with **zero unavailable responses**. It recorded timestamps and hashes. Four reported advisory aliases represent **two distinct issues**, avoiding inflated counts.

Affected local packages

Package / observed version	Advisory identifiers	Assessment and safe next step
certifi 2026.7.22	GHSA-6q48-6c7w-9gw2; PYSEC-2026-4320	Trust-bundle issue; upstream Critical, local High (12/25). Review upgrade to 2026.10.19+, establish client use and retest TLS validation.
pip 26.1.2	CVE-2026-13346; GHSA-qwm4-qh6w-59xr; PYSEC-2026-3721	Untrusted package-index URL issue; local Medium (6/25). Review upgrade to 26.2.0+, use approved indexes and retest in isolation.

What a match does not mean

• A registry advisory indicates the listed installed release matched a published issue; it is **not** proof of actual exploitation or a compromised system.

• The certifi upstream severity is distinct from DeRexi's **local** assessed impact; on-path conditions and use of that specific trust bundle were not demonstrated.

• The pip finding affects developer tooling and untrusted package-index conditions; it is **not** automatically an externally reachable DeRexi API defect.

• The audit did not verify deployed package versions, operating-system CVEs, container images, unknown vulnerabilities, or a complete software bill of materials.

Preserved integrity: no dependency was upgraded, no exploit was attempted, and no advisory finding was closed. The two matches were added to the same five-item OPEN remediation register.

07 / FINDINGS

Advanced control assessment & risk register

Failed tests are grouped by underlying security weakness rather than counted as separate vulnerabilities.

The final inspector identifies **five** OPEN findings: one Critical, two High and two Medium. Severity follows the local **likelihood × impact** model (maximum 25). It does not equal CVSS scoring.

Finding ID	Severity	Score	Root issue / affected asset
DVI-POLICY-WRITE	Critical	20/25	Four unauthorized policy-write assertions / APP
DVI-FEED-INTEGRITY	High	12/25	Insufficient FDIC URL/source binding / PULSE
DVI-ADVISORY-CERTIFI	High	12/25	Local vulnerable trust-bundle version / PKGS
DVI-DEPENDENCY	Medium	9/25	Unpinned dependency constraints / PKGS
DVI-ADVISORY-PIP	Medium	6/25	Local vulnerable tooling release / PKGS

Why policy-write is the first remediation priority

The assessment expected unsigned callers, Employees and non-approved roles to be denied privileged policy creation or publication. The tested handlers returned **201 Created** instead of 401/403, and the fake session recorded attempted inserts. In a governance product, unauthorized approved policies can change instructions employees trust. **Note:** the role-specific approval criterion requires business sign-off before it becomes an authoritative production rule.

Why feed attribution matters

A URL starting with **https** is not sufficient to establish that a story actually comes from FDIC. The negative test passed an unrelated host with an FDIC source label and the normalization function accepted it. The mitigation is publisher-host binding plus redirect validation, without disabling legitimate official/GovDelivery source routes.

Why dependency pinning matters

A lower-bounded requirements file permits future installations to resolve different library versions. Codex captured an exact runtime inventory but did not establish a reviewed, hash-verified lockfile or reproducible deployment. This is a software-supply-chain assurance weakness—not itself a published CVE.

08 / REMEDIATION

Treatment, retesting & ownership

An OPEN finding remains OPEN until the original fault is corrected and independently re-tested.

Finding	Proposed action	Closure evidence required
Policy writes	Add verified identity / server-side role dependencies to create/version handlers; agree approval policy	Denied signed Employee/Owner and anonymous writes with no recorded insert; permitted administrator workflow succeeds
Pulse host integrity	Bind reported source to allow-listed approved publisher hosts; validate redirects	Host-spoof rejection + legitimate publisher acceptance + current feed regression
Dependency reproducibility	Review lockfile and version/hash policy; generate deployable SBOM	Clean install from lock, consistency check and advisory triage
certifi advisory	Update reviewed CA bundle; keep TLS verification enabled	Version no longer affected; client trust-path verification
pip advisory	Upgrade reviewed pip version; approved indexes only	Fixed version matched; isolated approved-index operations

Workflow status recorded

Three originally identified findings were **discovered → validated → prioritized → tracked → retested still OPEN**. Two advisory findings were **discovered → registry-matched → triaged → tracked**, with the remediation retest pending. Original intake timestamps and SLA targets were retained. All proposed owners still have **acceptance pending**.

> **Do not equate a retest with a fix:** the independent retest reproduced the same failures. That strengthens evidence but does not reduce the exposure. Confirm the relevant production launch profile and real database behavior before any final sign-off.

Remediation accounting

5	0	0
Open	Closed	Approved exceptions

09 / RUBRIC

Official rubric traceability

Each course requirement, the actual local deliverable, and the boundary of what has been demonstrated.

Requirement	Evidence demonstrated	Accurate status
Enterprise vulnerability management program	Program governance, roles, evidence-handling, risk scoring, exception and escalation SOP	LOCAL workflow COMPLETE / ENTERPRISE PARTIAL
Advanced discovery, mapping and impact	13-asset reconciled inventory, dependency relationships, criticality and limitations	LOCAL COMPLETE / ENTERPRISE PARTIAL
Automated testing implementation	Repeated 33-check control suite; signed fixtures, logs and results	LOCAL COMPLETE
Custom deployed assessment tool	Inspector CLI executed, JSON/MD findings exported and nine regression tests	LOCAL COMPLETE
Advanced assessment and reporting	Six failed assertions, five triaged root findings, risk owners and closure tests	LOCAL COMPLETE
Performance metrics and verification	Coverage denominators, 35 release queries, retest and remediation counts	LOCAL COMPLETE / enterprise MTTR UNKNOWN

Scope-specific limitations

• The main tests use a fake database session; they cannot prove production SQL transaction behavior, Supabase RLS or external authentication issuer enforcement.

• The assessment does not show a fully implemented enterprise program, approved owners, recurring monitoring evidence, production inventory, or closed remediation.

• The documented Docker evidence was reused from an earlier run; it was not newly observed across a live production deployment.

Conclusion: substantial, defensible local technical delivery across the rubric. Overall enterprise-wide completion remains **PARTIAL** until the instructor accepts the local simulation as sufficient for the course scope, or broader controls are implemented and verified.

10 / EVIDENCE GUIDE

Reproduce, interpret & preserve the work

A practical map for revisiting the tools and defending the results during a demonstration.

Core source and execution files

Evidence	Purpose
program.md	Governance, personnel functions, severity, SLAs and closure policy
assets.json / assets-and-impact.md	Original source/evidence inventory and dependency diagram
closure/assets-reconciled.md + .json	13/13 reconciled assets with explicit provenance and limits
run_assessment.py / control_suite.py	Automated check runner and control expectations
runs/20261109T125144008620Z/	Final retest: execution metadata, controls logs, results, hashes, dependencies
derexi_vulnerability_inspector.py / test_inspector.py	Custom findings aggregator and its regression suite
closure/inspector/findings.json + findings.md	Final five-finding machine/human summaries
closure/advisories/assessment.json + advisories-triaged.json	35 PyPI version queries and deduplicated advisory triage
remediation-tracking.csv / closure/workflow.json	Five OPEN corrective actions and lifecycle events
closure/README.md / closure/commands.txt	Final rubric matrix, instructions and command transcript

Suggested explanation for an instructor

“I built a **repeatable, defensive vulnerability-management cycle** for DeRexi. The runner checks controls and preserves test evidence. My inspector groups failures into actionable findings, prioritizes them by business impact, and records exactly what must pass before closure. I reconciled 13 assets, directly assessed six, ran 33 controls, and performed 35 dependency release checks. I have not claimed that the five open findings were fixed or that a local test proves production security.”

11 / CLOSURE

Learning notes & next actions

Keep this page as your handoff from coursework evidence to the DeRexi final MVP.

Five concepts to remember

- **Inventory reconciliation ≠ control coverage.** Knowing where an asset appears is not the same as testing its defenses.
- **Failed assertion ≠ failed project.** A carefully reproduced negative test can demonstrate a meaningful vulnerability.
- **Finding ≠ check.** Four policy-write failures can represent one shared root cause.
- **Risk rating ≠ CVSS.** Local likelihood × impact is a decision tool, not a standardized exploit severity score.
- **Tracked ≠ remediated.** Only a correct change plus a clean, independent retest justifies closure.

Prioritized engineering handoff

Sequence	Next work	Evidence for completion
1	Lock down policy create/version/approval authorization and resolve business rule	Employee, Owner, anonymous denied; authorized admin accepted; live DB verification
2	Correct Pulse publisher/host binding and redirect safety	Spoofed host rejected without breaking verified feeds
3	Review/update certifi and pip safely in isolated environments	Fixed package versions and relevant client/tooling tests
4	Create reviewed dependency lock and software bill of materials	Reproducible clean install and dependency review
5	Expand control testing from six assets toward approved deployed scope	Real inventory, production issuer/RLS and repeated operational checks

Sources: uploaded Assignment 2 program, execution logs, asset and control records, inspector outputs, PyPI release JSON snapshots, and final closure documentation dated November 9, 2026. The document summarizes those materials and does not claim to have independently executed Codex's Windows assessment.